

Real-Time 3D SPH Fluid Simulation with Marching-Cubes Surface Reconstruction in WebGPU

Janez Koprivec¹

¹University of Ljubljana, Faculty of Computer and Information Science, Slovenia

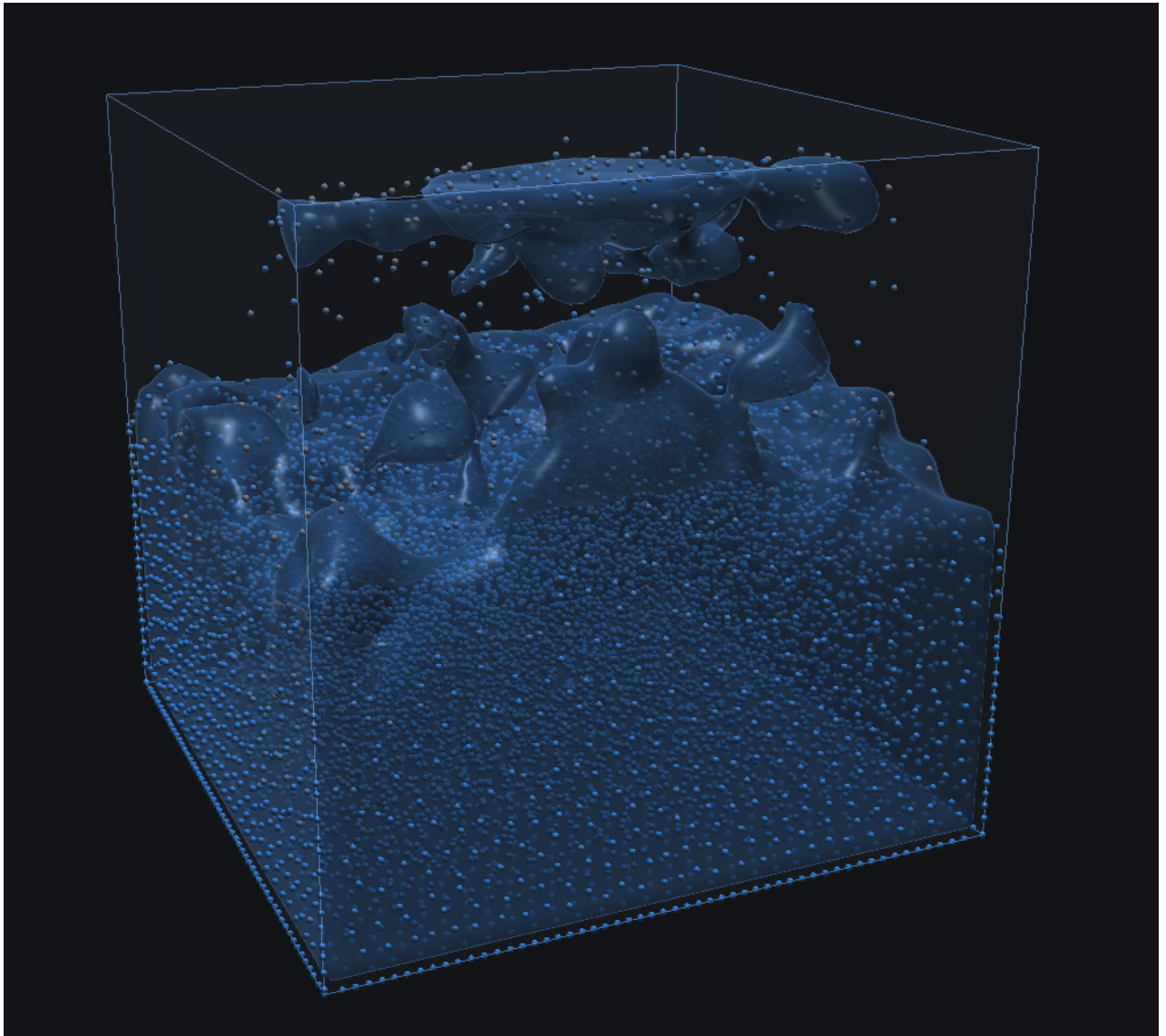


Figure 1: Pouring scenario at 10k particles, rendered simultaneously as point sprites (bulk fluid) and as a marching-cubes iso-surface of the SPH density field (smooth dome). The whole pipeline — neighbour search, SPH stages, density-field generation and marching-cubes extraction — runs on the GPU through WebGPU compute shaders.

submitted to COMPUTER GRAPHICS Forum (5/2026).

Abstract

This report describes a real-time 3D fluid simulator built around weakly compressible smoothed-particle hydrodynamics (WCSPH) and marching-cubes surface reconstruction. The solver, the neighbour search and the iso-surface extraction all run on the GPU through the WebGPU compute API, with a small CPU JavaScript SPH reference kept as an oracle for validation. The simulation reproduces the kernels and equation of state of [MCG03], accelerates neighbour search with a uniform spatial hash and a bitonic key sort, and renders the fluid either as point sprites or as a triangulated iso-surface generated each frame by a multi-pass marching-cubes pipeline (field build, classify, prefix scan, emit). The implementation runs in the browser, scales to roughly 10^4 particles in real time on commodity hardware, and was cross-validated against the CPU reference: bulk density agrees within $\sim 1\%$ over a 600-step run despite parallel-reduction-induced per-particle drift. We discuss the wall-handling heuristic that suppresses the SPH kernel-deficiency artefact near the boundary, the parameter sensitivity of WCSPH, and the realism ceiling of the explicit solver.

CCS Concepts

•Computing methodologies → Physical simulation; Rendering; Mesh geometry models;

1. Introduction

Fluid simulation is one of the recurring set-pieces of computer graphics: it shows up in offline VFX, interactive games and engineering visualisations, and it is hard for the same reason in all three settings — the Navier–Stokes equations are non-linear, intrinsically multi-scale, and produce qualitatively different behaviour depending on how their constraints (incompressibility, viscosity, boundary conditions) are discretised.

Smoothed-particle hydrodynamics (SPH) is a Lagrangian discretisation of the governing equations: the fluid is represented as a set of moving particles, each carrying mass and physical quantities such as velocity, density and pressure. Field quantities at any point in space are recovered by convolving the particle data with a smoothing kernel [Mon92]. Because particles *are* the fluid, mass is trivially conserved, free surfaces appear without any front-tracking, and the simulation domain does not need a global grid. The trade-off is that pressure, viscosity and surface terms must all be expressed as kernel sums over neighbours, and that the time step is limited by the speed at which information propagates through the particle set (the CFL condition on the SPH sound speed).

The seminar topic is exactly this method, in 3D, with surface reconstruction by marching cubes for visualisation. The scope covers:

- a basic SPH solver covering density estimation, pressure, viscosity and gravity;
- rendering the resulting fluid via surface reconstruction (marching cubes was the suggested choice);
- evaluation of the resulting system with respect to particle count, initial conditions and parameter choices.

The contribution of this seminar is a complete working implementation that runs in a browser via WebGPU, including: a weakly compressible SPH solver with poly6 / spiky / viscosity kernels and the Tait equation of state [MCG03, IOS*14]; a uniform spatial hash with a bitonic key sort for $O(N \cdot \bar{k})$ neighbour queries [HKK07]; a fully GPU-resident marching-cubes pipeline [LC87] that scans the iso-surface contribution per voxel and emits a compacted triangle mesh; and a CPU JavaScript SPH reference used as a numerical oracle. The whole pipeline reaches interactive frame rates

for $\sim 10^4$ particles on commodity hardware and was used to generate the figures throughout this report. The full source code, including the build instructions and the scenarios shown in the figures, is released under the MIT licence at <https://github.com/janezkoprivec/fluid-dynamics-3d>.

The remainder of this report is organised as follows. Section 2 reviews the WCSPH formulation we implemented and the marching-cubes surface extraction. Section 3 describes the GPU pipeline, the data layout, the neighbour search and the surface stage. Section 4 reports CPU/GPU parity, visual results and parameter sensitivity. Section 5 discusses the limitations of the explicit solver and Section 6 concludes.

2. Background

2.1. SPH formulation

In SPH, any continuous field $A(\mathbf{x})$ is reconstructed from the values A_j carried by neighbouring particles j at positions $\mathbf{x}_j$, weighted by a smoothing kernel $W(\mathbf{r}, h)$ of compact support h (the *smoothing radius*) [Mon92]:

$$A(\mathbf{x}) \approx \sum_j \frac{m_j}{\rho_j} A_j W(\mathbf{x} - \mathbf{x}_j, h). \quad (1)$$

The density is recovered as the special case $A_j = \rho_j$, which collapses (1) to

$$\rho_i = \sum_j m_j W(\mathbf{x}_i - \mathbf{x}_j, h). \quad (2)$$

Following Müller et al. [MCG03], we use three distinct kernels for different terms: the smooth C^2 poly6 kernel for density,

$$W_{\text{poly6}}(r, h) = \frac{315}{64\pi h^9} (h^2 - r^2)^3, \quad 0 \leq r \leq h, \quad (3)$$

the *spiky* kernel whose gradient is non-zero at $r = 0$, for the pressure term,

$$\nabla W_{\text{spiky}}(r, h) = -\frac{45}{\pi h^6} (h - r)^2 \hat{\mathbf{r}}, \quad (4)$$

and the viscosity kernel whose Laplacian is strictly positive in $[0, h]$,

for the viscous diffusion of velocity,

$$\nabla^2 W_{\text{visc}}(r, h) = \frac{45}{\pi h^6} (h - r). \quad (5)$$

Pressure is closed by the *Tait equation of state*, which makes the fluid *weakly compressible* (WCSPH):

$$p_i = k \left[\left(\frac{\rho_i}{\rho_0} \right)^\gamma - 1 \right], \quad \gamma = 7, \quad (6)$$

where ρ_0 is the rest density and k controls the stiffness of the response. Using a stiffness much lower than that of real water keeps the time step manageable at the price of a few-percent compressibility, which is the standard WCSPH trade-off [IOS*14].

The momentum equation discretised in SPH form, including a symmetrised pressure term and Müller-style viscosity, becomes:

$$\begin{aligned} \mathbf{a}_i = & - \sum_j m_j \frac{p_i + p_j}{2\rho_j} \nabla W_{\text{spiky}}(\mathbf{r}_{ij}, h) \\ & + \mu \sum_j m_j \frac{\mathbf{v}_j - \mathbf{v}_i}{\rho_j} \nabla^2 W_{\text{visc}}(\mathbf{r}_{ij}, h) + \mathbf{g}. \end{aligned} \quad (7)$$

We then integrate with symplectic Euler: $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t \mathbf{a}_i$ followed by $\mathbf{x}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$, substepped to stay below the CFL bound dictated by the SPH sound speed $c = \sqrt{k\gamma/\rho_0}$.

2.2. Neighbour search

A naive evaluation of (2) and (7) is $O(N^2)$, which is prohibitive for any reasonable particle count. SPH neighbour searches universally bin particles into a uniform grid of cell size h , then look up only the 27 cells of the local 3D neighbourhood [IOS*14, HKK07]. On GPU the canonical implementation is the *spatial-hash sort*: each particle emits a (cell, index) key, the keys are sorted by cell, and a separate pass fills a per-cell (start, end) table so that the SPH kernels only loop over a small, dense slice of particles.

2.3. Surface reconstruction by marching cubes

For visualisation, the particle cloud has to be turned into a surface. The standard SPH choice is to splat each particle into a regular 3D scalar field $\phi(\mathbf{x})$ using the same poly6 kernel, then extract an iso-surface at a level $\phi = \phi_{\text{iso}}$ (typically $0.5\rho_0$, which corresponds to the centre of the kernel-smoothing band of the surface particles). The iso-surface is meshed by the classical *marching-cubes* algorithm [LC87], which classifies each voxel of the field into one of 256 topological cases (based on which corners exceed the iso value) and emits zero to five triangles per voxel using a 256-entry lookup table. The result is a triangle mesh that can be rasterised by a standard shaded pass.

3. Implementation

3.1. Stack and architecture

The simulation is written in TypeScript and runs in a modern browser via the WebGPU API. All compute kernels are written in WGSL. The user interface uses Tweakpane for live parameter editing and stats.js for the FPS counter. The full pipeline is laid out as

a set of GPU compute passes orchestrated from a small host-side wrapper, plus two render passes (point sprites and triangulated iso-surface). A CPU JavaScript SPH solver lives next to the GPU one and is used exclusively as a numerical oracle for validation.

Figure 2 summarises the per-frame pipeline.

```
sim.step
  1. Hash particles into spatial-hash entries
  2. Bitonic sort entries by cell id
  3. Clear and build cellStart/cellEnd table
  4. Density + pressure (poly6, Tait EOS)
  5. Forces (spiky gradient + viscosity + gravity)
  6. Integrate + boundary handling
surface.update
  7. Field build (poly6 splat into 3D grid)
  8. MC classify (8-corner case index + tri count)
  9. Prefix scan (block-wise scan of triangle counts)
  10. Write indirect draw args
  11. MC emit (interpolated vertices + gradient normals)
renderer.draw
  12. Box wireframe
  13. Point sprites and/or triangulated iso-surface
```

Figure 2: Per-frame GPU compute and render pipeline. Steps 1–6 are the SPH solver, 7–11 the marching-cubes surface reconstruction, 12–13 the rendering. All compute passes are encoded into a single command buffer per frame.

3.2. Particle storage

Each particle is a 64-byte struct in a single GPU storage buffer:

```
position : vec3<f32>, _pad,
velocity : vec3<f32>, _pad,
acceleration : vec3<f32>, _pad,
density : f32, pressure : f32, _pad2.
```

The padding makes the natural 16-byte alignment of `vec3<f32>` in WGSL explicit, which removes a category of subtle layout bugs and keeps the struct shareable between every compute shader in the pipeline. The buffer is allocated for a fixed maximum particle count and resized only on explicit `sim.reset(N)`.

3.3. Spatial hash and bitonic sort

The spatial-hash pipeline mirrors the structure of [HKK07]. For each particle, the `hash.wgsl` pass writes a (`cellId`, `particleIndex`) pair into a key buffer (`hashEntriesA`). A bitonic sort then sorts these keys by `cellId` into `hashEntriesB` so that all particles in the same cell occupy a contiguous range. Finally, `cellTable.wgsl` scans the sorted array and writes per-cell (`start`, `end`) offsets into two N_{cells} -long arrays.

A bitonic sort was chosen over the more performant GPU radix sort because it is short (~ 100 lines of WGSL), self-contained, and fast enough at the target particle counts. All stages share one pipeline; the (j, k, mode) tuple for each stage is read from a single uniform buffer through a dynamic offset, which sidesteps the

trap that `queue.writeBuffer` between encoded passes is not sequenced by the GPU.

3.4. Density, forces, integration

The three core SPH compute passes are direct translations of (2), (7) and symplectic Euler into WGSL. They all share the layout in Figure 2: a read-only neighbour bind group (`sortedEntries`, `cellStart`, `cellEnd`), a uniform buffer with the simulation parameters, and a read-write storage buffer for particles.

The `density.wgsl` pass loops over the 27 neighbouring cells of the current particle’s cell, fetches the dense particle range from `cellStart/cellEnd` for each cell, evaluates the poly6 kernel, accumulates ρ_i , computes the Tait pressure (6), and clamps to a configurable $p_{\max}$ to avoid explosions on transient compression spikes. The `forces.wgsl` pass uses the same 27-cell traversal to evaluate both pressure (with the symmetric gradient form $p_{ij} = (p_i + p_j)/(2\rho_j)$) and viscosity, then adds gravity scaled by density to match the Müller momentum convention. The result is stored as the particle’s acceleration. The `integrate.wgsl` pass advances velocity then position, applies the wall handling described in the next subsection, and runs at a substepped inner time step capped at $\Delta t \leq 1/480$ s (a hard-coded constant chosen empirically to stay inside the CFL bound at the default h and k).

3.5. Boundary handling

A plain reflective box wall is not enough in SPH: a particle pressed against a wall sums its density over a sphere that is only half-filled with fluid neighbours (the other half is outside the wall), under-reports ρ , under-shoots p through (6), and is therefore pushed harder *into* the wall by bulk pressure than it can push back. Without compensation, the SPH bulk pressure drives the fluid into the boundary until it stratifies into horizontal sheets along the floor and walls (Figure 3); once stuck the fluid stays stuck, because each compressed sheet acts as a new kernel-deficient “wall” for the layer above it.

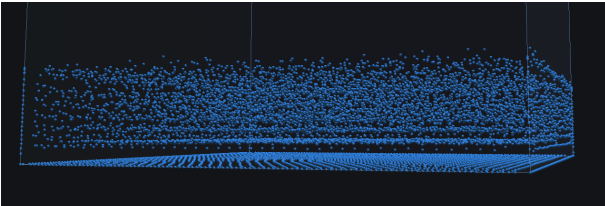


Figure 3: Wall-band parameters set to zero. The kernel deficit at the boundary lets bulk pressure pile the fluid into stable horizontal sheets along the floor and the lower walls. Restoring the wall band breaks this regime.

The proper fix is ghost or boundary particles; the cheap fix that works visually is a Monaghan-style soft repulsion band. Within a band of width $\eta \cdot h$ of any wall ($\eta = 0.7$), an outward acceleration $a = Kt^2$ is added, where $t \in [0, 1]$ ramps from 0 at the band edge to 1 at the wall and K is the repulsion strength. To stop the band from behaving as a purely conservative spring (which would make every

wall bounce springily), a velocity-normal damping $a' = -cv_n t$ is also applied, but only when the velocity component points *into* the wall so it can never trap a particle. The reflective wall is kept as a hard backstop for the rare cases where a particle still tunnels at high velocity. Figure 4 shows the resulting steady state with the band active.

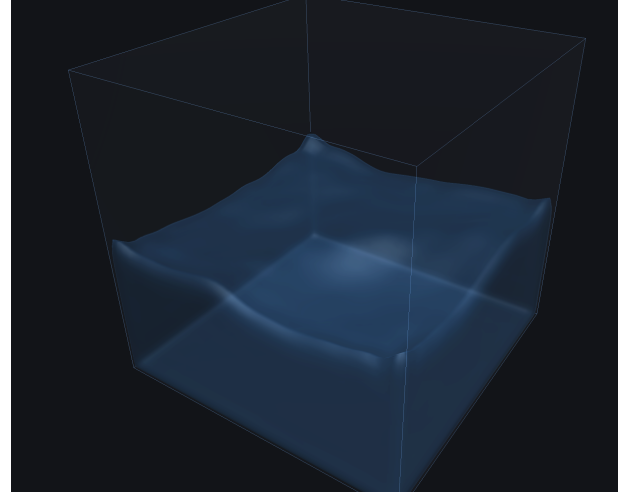


Figure 4: Settled fluid with the wall band active. The flat top and gentle corner curvature show that the band suppresses the kernel-deficit layering of Figure 3 without over-damping the bulk.

3.6. Marching-cubes surface reconstruction on GPU

The marching-cubes pipeline runs entirely on the GPU as a sequence of compute passes orchestrated from `surface/index.ts` (seven distinct compute pipelines; the per-frame dispatch count depends on the grid resolution, since the prefix-scan stage is hierarchical). **Field build** dispatches one thread per grid vertex of an $n_x \times n_y \times n_z$ scalar field; each thread loops over the active particles and accumulates their poly6 contribution at its sample position. The gather direction (thread-per-voxel) avoids the atomics that a scatter implementation would require, and voxels outside the simulation box are clamped to $\phi = 0$ so marching cubes closes the iso-surface flush against the glass. **Classify** samples the eight corners of each voxel, builds the 8-bit case index against ϕ_{iso} , and looks up the triangle count from a 256-entry table uploaded once at startup.

The write offset of each voxel in the output vertex buffer is the *exclusive prefix sum* of triangle counts up to that voxel. We compute it with a Blleloch-style multi-level scan (`scanLocal`, `scanBlockSums`, `scanAddOffsets`) that is a generic GPU compaction primitive — tested standalone before being plugged in. A small `computeDrawMeta` pass reads the final element of the prefix and triangle-count buffers, clamps to `maxTriangles`, and writes the four-word `drawIndirect` arguments directly into a GPU buffer, so the CPU never learns the triangle count. The final **emit** pass reads each voxel’s case index, interpolates vertex positions along the cube edges and computes normals from central

differences of the scalar field, writing the result at the offsets reserved by the scan. The renderer then rasterises the mesh with a Phong+Fresnel shader and exposes *points*, *surface* and *both* modes so the iso-surface can be visually checked against the particles it should fit.

3.7. Interaction: a mouse-driven force wand

To make the demo something other than a fixed scenario, the user can push or pull the running fluid with the mouse. The cursor is unprojected into a world-space ray; we intersect this ray with the simulation box AABB (slab test) and take the entry point as the interaction anchor $\mathbf{x}_w$. When the user drags with the modifier key held, we additionally track a smoothed drag direction $\hat{\mathbf{d}}_w$ over the last few frames so individual cursor jitters do not produce spiky impulses.

The wand state is exposed to the GPU through a small `InteractionParams` uniform in its own bind group, written by the host every frame. Inside `forces.wgsl`, after gravity, we add an external acceleration on particles inside the wand radius:

$$\mathbf{a}_w = \rho_i \cdot s \cdot w(r) \cdot \begin{cases} \hat{\mathbf{d}}_w & (\text{push}) \\ -\hat{\mathbf{r}}/\|\mathbf{r}\| & (\text{pull}) \end{cases} \quad (8)$$

with $\mathbf{r} = \mathbf{x}_i - \mathbf{x}_w$, smooth falloff $w(r) = (1 - r/R)^2$ inside the radius R and zero outside, and a user-controlled strength s . Scaling by ρ_i matches the gravity convention already in the shader, so a given s produces the same visible acceleration regardless of local density. Modes (push vs. pull), radius, and strength are bound to GUI sliders.

Because the wand is just a per-frame external acceleration added in the same pass that already evaluates gravity, no new compute pass or neighbour traversal is introduced; the cost is a single uniform read plus a branch per particle. Figure 5 shows the wand in use — a vertical pull lifts a peak of fluid out of the settled pool.

A solid-body drop feature (a sinking/floating sphere) was also prototyped but descope: one-way coupling looked wrong (the sphere hovered above where the surface visibly was) and proper two-way coupling requires a per-frame GPU readback of accumulated contact forces that would have ballooned the scope of the seminar. The wand demonstrates fluid–structure interaction adequately for the deliverable.

3.8. Validation and parity harness

For validation we built a CPU JavaScript SPH solver with the same kernels, EOS and integrator. It deliberately mirrors the GPU code path field for field, so that a divergence in behaviour is almost always caused by GPU-specific issues (parameter packing, sort bugs, atomic ordering) rather than by physics differences. A parity harness in `src/app.ts` clones the CPU dam-break scenario onto the GPU, runs both sides at lockstep $\Delta t = 1/240$ s for 600 steps, and reads back the GPU buffer every 30 steps to compare $\min/\text{avg}/\max \rho$, $|\mathbf{v}|_{\max}$, KE and $|\mathbf{p}|$ against tolerance bands.

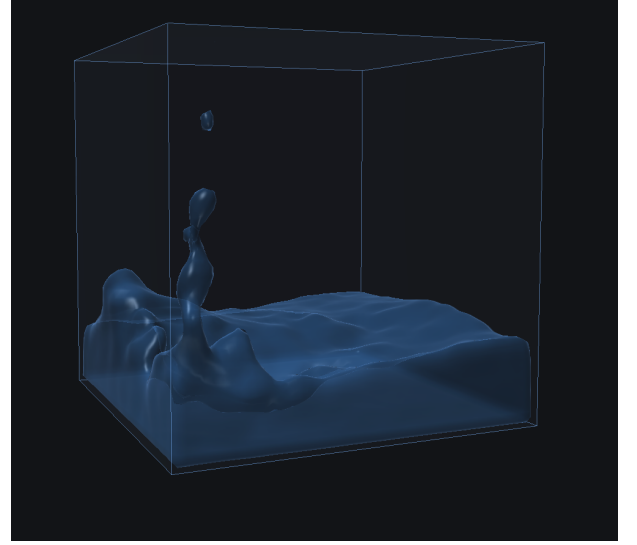


Figure 5: Wand interaction: a pull force centred above the pool lifts a peak of fluid out of the surface. The peak collapses into a droplet as the force is released.

4. Results

4.1. CPU/GPU numerical parity

The CPU and GPU solvers were stepped in lockstep on a 1536-particle dam-break at $\Delta t = 1/240$ s for 600 steps, with the GPU wall band, the boundary tangent damping and the CFL substep cap matched on both sides so that the only remaining difference is the SPH math itself. Table 1 reports the relative difference $|\text{CPU} - \text{GPU}|/\text{CPU}$ on five representative samples.

step	$\Delta \rho_{\text{avg}}$	$\Delta \mathbf{v} _{\max}$	ΔKE	$\Delta \mathbf{p} _{\max}$
30	0.47 %	0.00 %	0.07 %	0.40 %
60	0.06 %	1.07 %	0.00 %	2.17 %
90	0.10 %	10.47 %	3.08 %	1.53 %
300	1.43 %	28.06 %	3.15 %	50.16 %
600	0.04 %	50.97 %	57.92 %	79.59 %

Table 1: CPU/GPU relative difference on a 600-step dam-break, $N = 1536$, $\Delta t = 1/240$ s.

Across all 20 samples, ρ_{avg} stays within 3.63 % of the CPU oracle (median below 1 %). For the first ~ 90 steps, the entire metric set agrees within ~ 3 % — the period in which the system is still meaningfully “flowing”. After that, per-particle metrics like KE and $|\mathbf{p}|_{\max}$ start to drift substantially while the bulk density distribution keeps tracking. The cause is parallel-reduction-order chaos: the CPU sums the neighbour kernels sequentially, the GPU does tree reductions, and the resulting sub-ULP differences are amplified by the chaotic dynamics of the equilibrium fluid. The same effect appears in published GPU SPH parity studies and is what motivates statistical, not bit-exact, validation [IOS*14]. We take this as evidence that the GPU SPH math — density, pressure, viscosity, integration — is correct.

4.2. Visual results across scenarios

Figure 1 (front page) shows the system at the target operating point: a ~ 10 k-particle pour stream impacting a pool in the *both* render mode. Figure 6 shows the pure surface view of the same scenario mid-fill, where the marching-cubes mesh traces the stream and accumulating pool while detached droplets pop in and out of existence as the iso-surface condition turns on and off around them.

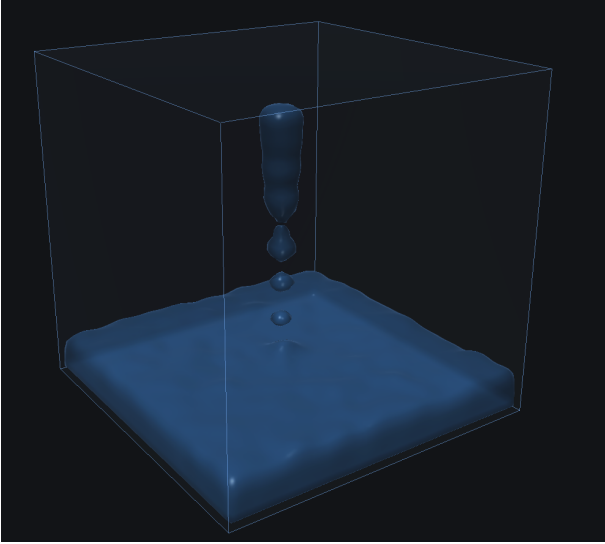


Figure 6: Mid-fill of the pouring scenario in pure surface mode. The marching-cubes mesh traces the continuous stream above the pool and resolves individual droplets that have broken off.

4.3. Parameter sensitivity

The two parameters with the most visible effect on the look of the fluid are the viscosity coefficient μ and the marching-cubes iso value ϕ_{iso} . At low μ (Figure 7) the splash on impact becomes visibly more energetic and individual droplets separate from the bulk, which is closer to real water but also closer to the stability boundary of the explicit solver: lowering μ much further leads to velocity spikes that get clamped by p_{max} . At high μ the fluid takes on a syrupy, smoothed-out look in which splashes are visibly under-damped.

The iso value controls where in the kernel-smoothing band the surface sits. At $\phi_{\text{iso}} \approx 0.5\rho_0$ the surface is the visually clean “halfway through the band” choice recommended by [MCG03]; lowering it below $\sim 0.3\rho_0$ produces a puffy, blocky surface that picks up the outer halo of the kernel; raising it past ρ_0 disconnects the surface into beads of fluid where the bulk doesn’t quite reach full rest density.

A failure mode worth documenting is what happens when the timestep, viscosity and gas constant fall out of the WCSPH stability triangle: the simulation explodes within milliseconds (Figure 8). The maximum-pressure clamp catches most cases, but a too-stiff k at a too-large Δt will always win against any clamp. The defaults

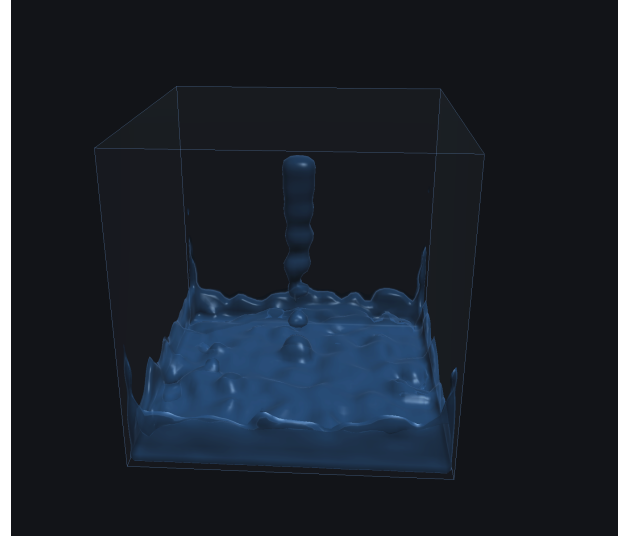


Figure 7: Low-viscosity pour ($\mu = 1.0$): the impact produces a clean central jet and rim of small droplets, visually closer to water. The trade-off is reduced stability margin.

in the released demo were tuned by bisecting around the recommended values from [MCG03] until a 30-second pour remained visually plausible.

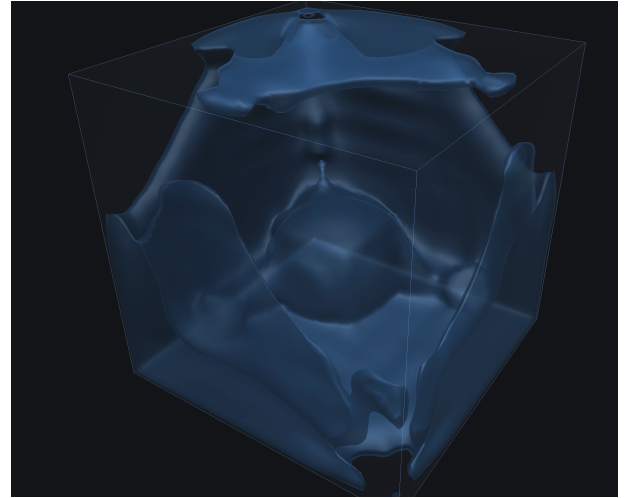


Figure 8: Catastrophic failure: a too-large timestep ($\Delta t = 1/60$ s, no substep cap) combined with the production stiffness gives a snapshot where the SPH sound speed has been exceeded and the iso-surface is being violently re-meshed every frame. Included to motivate the substep cap and pressure clamp.

4.4. Scaling and performance notes

The pipeline as implemented reaches interactive frame rates for ~ 10 k particles on a commodity laptop GPU. The dominant costs

are, in decreasing order: the field-build pass (it is $O(N_{\text{voxels}} \cdot N_{\text{particles}})$ in its current gather form), the density and forces passes (now reduced to $O(N \cdot \bar{k})$ with $\bar{k} \approx 35$ thanks to the spatial hash), and the bitonic sort (which is $O(N \log^2 N)$ but on small key sizes). Lifting the field-build cost would either route its sampling through the same spatial-hash table as the SPH passes, or switch to a particle-scatter implementation with atomics; both are listed as deferred work in the project’s planning notes and are not required for the seminar deliverable. The marching-cubes scan and emit passes scale linearly in the number of voxels and have not been the bottleneck at the grid resolutions tested (up to 64^3).

5. Discussion

The implementation hits the seminar target — a runnable, interactive 3D SPH solver with a marching-cubes surface — but several limitations are worth calling out, both because they shape the visual result and because they are interesting in their own right.

WCSPH compressibility. The Tait EOS allows a small amount of compression ($\sim 1\%$ in practice), which manifests as a faint “breathing” of the surface under load. Truly incompressible behaviour requires either a much larger stiffness k (and a proportionally smaller Δt) or a switch to one of the predictive-corrective family of solvers (PCISPH, IISPH, PBF) that add a pressure-correction loop inside each step [IOS*14]. We intentionally stayed with WCSPH because the scope is a basic SPH solver, and because the corrective schemes meaningfully complicate the GPU implementation.

Boundary handling. The soft band described in §3.5 is an ad-hoc fix for the kernel deficit at walls, and is the most visible source of non-physical behaviour. The principled fix is to seed a layer of boundary particles or ghost particles that mirror the fluid across each wall; with those in place the density at the wall is correctly computed and the bulk pressure stops driving particles into the boundary. We did not implement this because the band-aid is good enough for the seminar’s visual quality and stability targets, and because ghost particles interact awkwardly with the spatial-hash neighbour table.

Surface tension and droplet realism. WCSPH has no cohesion term, so the surface tension that holds droplets together in real fluids is absent here. Detached fluid does form droplet-shaped iso-surfaces because of the kernel-smoothing radius, but the droplet behaviour is purely a consequence of mass and pressure, not surface energy. The standard remedy is Müller’s follow-up cohesion force on a colour field, which we did not implement.

Surface aliasing at moderate grid resolutions. At grid resolutions below $\sim 48^3$ the marching-cubes surface shows visible step-shading artefacts where neighbouring voxels share field values that straddle the iso threshold differently. Raising the grid resolution to 64^3 removes most of them at the cost of an N^3 increase in field-build work; doubling the resolution is roughly the largest step the current field-build pass can absorb without dropping below interactive rates.

6. Conclusion

This seminar implemented a 3D SPH fluid simulator with marching-cubes surface reconstruction, runnable in a browser via WebGPU. The solver follows the WCSPH formulation of Müller et al. [MCG03], the neighbour search uses a uniform spatial hash with a bitonic key sort following Harada et al. [HKK07], and the surface stage runs the full marching-cubes pipeline of Lorensen & Cline [LC87] on the GPU as a sequence of seven compute passes ending with an indirect-drawn triangulated mesh. The implementation was cross-validated against a CPU JavaScript reference solver, and the resulting demo reaches interactive frame rates for $\sim 10^4$ particles on commodity hardware.

The deliverable suggests several natural extensions for follow-up work: routing the marching-cubes field build through the existing spatial-hash table to lift the particle-count ceiling, adding implicit viscosity to bring μ closer to that of real water, and replacing the soft-band boundary heuristic with proper boundary particles. These are deferred to a thesis-scope continuation of the project; the present report describes the seminar implementation as delivered.

References

- [HKK07] HARADA T., KOSHIZUKA S., KAWAGUCHI Y.: Smoothed particle hydrodynamics on GPUs. In *Proceedings of Computer Graphics International (CGI '07)* (2007), pp. 63–70. [2, 3, 7](#)
- [IOS*14] IHMSEN M., ORTHMANN J., SOLENTHALER B., KOLB A., TESCHNER M.: SPH fluids in computer graphics. *Eurographics 2014 – State of the Art Reports* (2014), 21–42. [doi:10.2312/egst.20141034.2, 3, 5, 7](#)
- [LC87] LORENSEN W. E., CLINE H. E.: Marching cubes: A high resolution 3D surface construction algorithm. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)* (1987), ACM, pp. 163–169. [doi:10.1145/37402.37422.2, 3, 7](#)
- [MCG03] MÜLLER M., CHARYPAR D., GROSS M.: Particle-based fluid simulation for interactive applications. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '03)* (2003), Eurographics Association, pp. 154–159. [doi:10.5555/846276.846298.2, 6, 7](#)
- [Mon92] MONAGHAN J. J.: Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics* 30 (1992), 543–574. [doi:10.1146/annurev.aa.30.090192.002551.2](#)